

Proposed Plan: Piecewise-Linear Variable Loss Curves

1. Recommended approach

Implement piecewise-linear losses using an **event-driven active-segment LP algorithm**.

For each loss location, the solver will:

1. Determine the currently active loss-curve segment.
2. Linearize the loss relationship for that segment.
3. Restrict the LP so the flow cannot leave the segment.
4. Maximize the current transaction or proportional group.
5. If the solution reaches a segment boundary, commit the allocation made so far.
6. Activate the adjacent segment, update the loss equations, and continue maximizing the same schedule item.
7. Stop only when the transaction is limited by a real supply, demand, group, or transaction constraint rather than by an internal loss breakpoint.

This preserves the existing continuous-LP and lexicographic structure without introducing binary variables.

A mixed-integer SOS2 formulation is another possible implementation, but the event-driven approach should fit the current solver better and should usually require fewer expensive solves than a full MIP.

2. Important mathematical distinction

A linear segment of an absolute-loss curve does not generally have a constant average loss fraction.

For one curve segment:

$$L(Q) = mQ + b$$

where:

- Q is flow immediately above the loss location.
- $L(Q)$ is the absolute loss.
- m is the segment's marginal loss rate.
- b is the segment intercept.

The flow remaining below the loss location is:

$$D(Q) = Q - L(Q) = (1 - m)Q - b$$

The segment has a constant **marginal loss rate** m , but the average loss fraction is:

$$\frac{L(Q)}{Q} = m + \frac{b}{Q}$$

Therefore, simply substituting the segment slope into the existing expression:

$$\text{downstream} = \text{upstream} * (1 - \text{loss_fraction})$$

would be incorrect whenever $b \neq 0$.

The intercept must be represented either by an aggregate loss equation or by carrying forward a baseline solution when the segment changes.

3. Use baseline-relative continuity equations

The current solver builds homogeneous transaction continuity equations such as:

$$v_2 = (1 - r)v_1$$

That works for a constant proportional loss because the loss curve passes through the origin.

For a general piecewise-linear segment, use incremental equations relative to the allocation already committed when the segment became active.

Let:

- u_0 be the transaction component above the loss when the segment is entered.
- d_0 be the corresponding component below the loss.
- u and d be their current total values.
- m be the active segment slope.

Then require:

$$d - d_0 = (1 - m)(u - u_0)$$

or equivalently:

$$d - (1 - m)u = d_0 - (1 - m)u_0$$

If transaction-specific losses are retained, the transaction's loss component can similarly satisfy:

$$\ell - \ell_0 = m(u - u_0)$$

This gives the intended procedural interpretation:

- Previously allocated water retains the losses assigned under earlier segments.
- New water allocated while the current segment is active receives the current marginal loss rate.
- When a breakpoint is reached, the current totals become the new baseline.

This is the central change needed in `solver.py`.

4. Extend `loss_models.py`

4.1 Add an explicit segment representation

Add a compiled segment class similar to:

```
@dataclass(frozen=True)
class LossSegment:
    index: int
    min_inflow: float
    max_inflow: float | None
    slope: float
    intercept: float

    def loss_at(self, inflow: float) -> float:
        return self.slope * inflow + self.intercept

    def delivered_at(self, inflow: float) -> float:
        return inflow - self.loss_at(inflow)
```

`LossCurve` should expose:

```
def get_segments(self) -> list[LossSegment]
def get_segment(self, inflow: float, direction: int = 0) -> LossSegment
def get_next_breakpoint(self, inflow: float, direction: int) -> float | None
```

The final capped-loss portion becomes an unbounded segment with:

$$m = 0, \quad b = L_{\text{final}}$$

4.2 Normalize constant fractions

Internally, convert a constant fraction r to a one-segment curve:

$$L(Q) = rQ$$

This lets the solver use one implementation for:

- Constant fractions
- Piecewise-linear curves
- Time-varying constant fractions
- Time-varying curves

4.3 Resolve time-varying losses once per day

Add a function such as:

```
def resolve_loss_definition(  
    definition: LossDefinition,  
    date: str,  
) -> LossCurve:  
    ...
```

`TimeVaryingLoss` should be resolved at the beginning of each day. The LP should then work only with the resolved static curve.

4.4 Add physical validation

For the first implementation, require:

- Nonnegative curve inflow.
- Nonnegative losses.
- Loss no greater than inflow.
- Continuous segments.
- Delivered flow $Q - L(Q)$ must be nondecreasing.

The last requirement means:

$$m \leq 1$$

Negative slopes may be allowed. For example, absolute loss may decline as flow increases, as long as the resulting delivered flow remains well-defined.

A segment with $m = 1$ represents 100% marginal loss. It can be supported when the pre-loss flow is explicitly represented, but it cannot generally be inverted from a downstream measurement. For a `loss_from_zone` curve inferred only from a post-loss gauge, such a segment is ambiguous and should produce a validation error.

5. Introduce a compiled `LossSite`

Do not let the rest of the solver manipulate `LossDefinition` objects directly. Compile each configured loss into an internal loss site.

Suggested structure:

```
@dataclass(frozen=True)
class LossSite:
    id: str
    flow_id: str
    side: Literal["from_zone", "to_zone"]
    direction: int
    loss_route_flow_id: str
    definition: LossDefinition
```

Daily state:

```
@dataclass
class ActiveLossState:
    site: LossSite
    curve: LossCurve
    segment_index: int
    baseline_driver_flow: float
    baseline_total_loss: float
    baseline_component_values: dict[str, float]
```

The compiled loss site should identify:

- Where the loss occurs.
- The flow expression immediately above the loss.
- The physical route receiving the loss.
- The active direction.
- Which transaction components cross the site.
- Which downstream transaction components are affected.

This will also centralize the currently scattered `loss_from_zone` and `loss_to_zone` direction logic.

6. Define the loss-driver expression explicitly

Each loss site needs a linear expression representing total flow immediately above the loss:

$$Q_s = \sum_i a_i x_i$$

where x_i are transaction components and the coefficients reflect physical direction.

The meaning of the driver must be explicit. Possible drivers include:

- Total flow on an observed interzone flow.
- Reconstructed pre-loss flow above a gauge.
- Total inflow to a zone.
- Remaining stream flow after upstream allocations.
- A directional component of a bidirectional flow.

This distinction matters because a loss based directly on a fixed daily measurement will have a fixed active segment. It will not change as transaction labels are allocated.

The event-driven segment logic is needed when the driver represents a quantity that changes during allocation, such as remaining zone inflow or reconstructed flow above a diversion.

For the initial implementation, I recommend limiting variable curves to nonnegative, unidirectional driver expressions. Bidirectional loss curves require either:

- Separate forward and reverse loss sites, or
- Positive and negative flow decomposition with additional logic preventing simultaneous counterflow.

7. Add aggregate loss constraints

For each site, add an aggregate loss variable:

LOSS_TOTAL_<site_id>

On active segment j :

$$L_s - m_j Q_s = b_j$$

Add a constraint:

LOSS_CURVE_<site_id>

with coefficients representing:

$$L_s - m_j Q_s = b_j$$

Also add segment guard constraints:

$$Q_{\min,j} \leq Q_s \leq Q_{\max,j}$$

These can use a single ranged constraint:

```
LOSS_RANGE_<site_id>
```

The aggregate equation is the authoritative physical relationship. Transaction-specific loss components, when retained, must sum to `LOSS_TOTAL_<site_id>`.

This prevents component-level rounding or attribution choices from violating the physical curve.

8. Refactor transaction-specific loss variables

The current `_expand_path_with_losses()` adds another `TrxnPathItem` using the physical loss-route flow ID. This becomes fragile with multiple loss sites and does not distinguish two losses routed through the same interzone flow.

Instead, create dedicated transaction loss variables:

```
LOSS_ALLOC_<site_id>___<transaction_id>
```

For each transaction crossing the site:

$$\ell_i - \ell_{i,0} = m_j(q_i - q_{i,0})$$

Then require:

$$L_s = \sum_i \ell_i$$

The transaction's downstream component satisfies:

$$d_i - d_{i,0} = (q_i - q_{i,0}) - (\ell_i - \ell_{i,0})$$

This is clearer than multiplying several loss fractions into one `rem_factor`, and it supports:

- Nonzero segment intercepts
- Multiple loss locations between observations
- Transaction-level loss reporting
- Aggregate physical validation
- Segment changes without rewriting historical allocations

As a smaller first milestone, aggregate loss can initially be represented as a system-loss transaction. Transaction-specific marginal attribution can then be added after the active-segment engine is working.

9. Make loss constraints dynamically updateable

The LP wrapper already supports changing coefficients and bounds. Add higher-level methods such as:

```
def replace_constraint(
    self,
    name: str,
    coefficients: dict[str, float],
    lb: float | None,
    ub: float | None,
) -> None:
    ...
```

and:

```
def set_constraint_bounds(
    self,
    name: str,
    lb: float | None,
    ub: float | None,
) -> None:
    ...
```

The loss-state manager should be solely responsible for updating:

- Segment slopes
- Segment intercepts
- Baseline-relative continuity right-hand sides
- Segment flow bounds
- Transaction-specific marginal loss coefficients

Avoid rebuilding the entire `Apportioner` at every breakpoint.

Because the SciPy wrapper reconstructs the sparse matrix for every solve, this will still trigger a fresh numerical solve. Once the algorithm is correct, the `highspy` wrapper could be used to retain a simplex basis and make the repeated segment solves substantially faster.

10. Add an event-driven objective runner

Wrap the existing `_maximize_var()` and proportional-group solve logic in a loss-aware loop.

Conceptually:


```

def maximize_with_loss_events(objective):
    while True:
        loss_manager.apply_active_segments(engine)

        baseline = capture_current_state()
        solution = solve_current_objective(objective)

        events = loss_manager.find_boundary_events(
            baseline=baseline,
            solution=solution,
        )

        commit_progress(solution)

        if not events:
            finalize_objective(objective)
            return solution

        for event in events:
            loss_manager.advance_segment(event)

        loss_manager.capture_new_baselines(solution)

```

10.1 Boundary detection

For each loss site, compare:

$$\Delta Q_s = Q_{s,\text{solution}} - Q_{s,\text{baseline}}$$

A boundary event occurs when:

- Q_s reaches the active segment's lower or upper bound within tolerance.
- An adjacent segment exists in the direction of movement.
- The segment bound is preventing further improvement.

If several sites reach breakpoints simultaneously, advance all of them before the next solve.

10.2 Starting exactly on a breakpoint

When the baseline is exactly on a breakpoint:

- Prefer the segment corresponding to the direction in which the objective moves the driver.
- If the first solve makes no progress because the selected segment guard is tight, try the adjacent segment.
- Do not report the temporary segment guard as the transaction's final limiting constraint.

10.3 Prevent cycling

Track:

```
(site_id, segment_index, direction, objective_id)
```

during the objective solve.

Raise a diagnostic error if the same state is repeatedly revisited without objective improvement. Also impose a conservative event limit derived from the total number of curve segments.

11. Preserve lexicographic priority correctly

Changing active segments changes the LP representation even though it represents the same physical piecewise-linear problem. Therefore, relying only on lower bounds for completed senior transactions is risky.

Use two types of commitment:

Progress commitment

While the same transaction or proportional group continues through another segment:

```
lb = current_value
```

The variable may continue increasing.

Final priority commitment

Once a schedule item is genuinely maximized:

```
lb = final_value  
ub = final_value
```

or add an equality constraint fixing its objective value.

This ensures a senior allocation cannot change when later segment equations are activated or junior transactions are solved.

For proportional groups:

- Maintain the achieved group proportion while traversing loss segments.

- Fix a member when it becomes genuinely bottlenecked.
 - Continue the existing proportional-series logic for the remaining members.
 - Treat breakpoint events as internal continuation events, not as members becoming maximized.
-

12. Refactor natural-flow calculations

The following current operations assume losses are numeric fractions and will all need replacement:

- `DailyDataManager.calc_natural_flows()`
- `DailyDataManager._get_calculated_losses()`
- `Apportioner._build_linear_equations()`
- `Apportioner.apply_nf_mass_balance_constraints()`
- `TrxnManager._expand_path_with_losses()`

Recommended staged implementation

Stage A: Exogenous-flow curves

First support piecewise-linear losses whose driver is a measured or otherwise fixed daily flow.

For each day:

1. Resolve the loss definition.
2. Evaluate the loss amount directly using `get_loss_amount()`.
3. Determine the active segment before building the daily LP.
4. Use the exact daily loss in natural-flow propagation.

This stage supports time-varying and piecewise-linear curves but does not yet require breakpoint events during allocation.

Stage B: Allocation-dependent curves

Then support losses based on changing zone inflow or remaining stream flow.

For these sites:

- Represent the natural-flow availability and loss relationships inside the LP.
- Update the applicable natural-flow constraints after every loss event.
- Do not treat `available_natural` as one fixed number calculated before the LP.
- Carry the same loss-state engine through both the natural-flow pass and spill-reallocation pass.

The spill pass should remove or alter only the intended natural-flow restrictions. It should never remove loss-curve equations or segment guards.

13. Handle `loss_from_zone` and `loss_to_zone` separately

For `loss_to_zone`, the observed interzone flow is above the loss:

$$Q_{\text{above}} = Q_{\text{observed}}$$

and:

$$Q_{\text{below}} = Q_{\text{observed}} - L(Q_{\text{observed}})$$

For `loss_from_zone`, the observation is below the loss:

$$Q_{\text{observed}} = Q_{\text{above}} - L(Q_{\text{above}})$$

On active segment j :

$$Q_{\text{observed}} = (1 - m_j)Q_{\text{above}} - b_j$$

When $m_j < 1$:

$$Q_{\text{above}} = \frac{Q_{\text{observed}} + b_j}{1 - m_j}$$

When $m_j = 1$, the observed downstream flow does not identify the pre-loss inflow. This is the zero-flow/100%-loss ambiguity previously discussed. Such a curve should either:

- Be driven by explicit zone inflow, or
- Be rejected for a post-loss-only measurement configuration.

The direction handling should be implemented in one `LossSite` compiler rather than independently in natural-flow propagation and transaction continuity.

14. Update static transaction compilation

`TrxnManager` is created before the daily loop, while `TimeVaryingLoss` is resolved by date. Therefore, transaction path structure cannot depend on whether a loss happens to be zero on one particular day.

Compile all potentially active loss sites statically. Resolve only their coefficients and active status daily.

Replace comparisons such as:

```
if l_from > 0:
```

because `LossCurve` and `TimeVaryingLoss` cannot be compared to a number.

Use functions such as:

```
has_configured_loss(definition)
resolved_loss_is_zero(definition, date)
```

Also avoid permanently mutating the user's original transaction paths. Build compiled transaction/path objects for the solver instead.

15. Evidence and audit output

Add loss-event evidence to the solve output:

```
@dataclass
class SolverOutputLossEvent:
    date: str
    solve_phase: str
    objective_id: str
    loss_site_id: str
    old_segment_index: int
    new_segment_index: int
    boundary_flow: float
    flow_before: float
    flow_after: float
    old_slope: float
    new_slope: float
```

For each transaction, retain evidence showing:

- Marginal loss rate used for each allocation increment.
- Loss amount attributed to the transaction.
- Breakpoints crossed during its allocation.
- Final physical constraint that stopped the allocation.

A segment boundary should appear as a continuation event, not as the final explanation for why a transaction received less water.

16. Testing plan

`loss_models.py` unit tests

Test:

- Automatic insertion of `(0, 0)`.
- Segment slope and intercept calculations.
- Final capped-loss segment.
- Exact values at breakpoints.
- Selection immediately above and below a breakpoint.
- Negative loss slopes.
- Marginal slope equal to one.
- Rejection of slopes greater than one, if monotonic delivery is required.
- Time-varying interval selection.
- Missing-date behavior.
- Constant-fraction normalization.

Solver tests without breakpoint crossings

- Constant loss regression tests produce unchanged results.
- A piecewise curve operating entirely within one segment.
- A segment with a nonzero intercept.
- A capped-loss final segment.

Breakpoint-event tests

- A diversion decreases the loss driver through one breakpoint.
- An import increases the driver through one breakpoint.
- One transaction crosses several breakpoints.
- Two loss sites reach boundaries during the same solve.
- The starting flow is exactly at a breakpoint.
- The objective ends exactly at a breakpoint.
- A proportional group crosses a breakpoint together.
- A junior transaction starts in the segment left by a senior transaction.
- A breakpoint is crossed during spill reallocation.

Physical edge cases

- A low-flow 100%-loss segment followed by a lower marginal-loss segment.
- Zero downstream measurement with a non-unique pre-loss inflow.
- A decreasing absolute-loss curve.
- Time-varying curves changing between days.
- A curve on a bidirectional flow is rejected until directional support exists.
- Multiple losses between two transaction path observations.

Oracle comparison

For small test networks, enumerate every possible combination of active segments and solve the corresponding LPs. Compare the event-driven result with the best feasible lexicographic result from this enumeration.

This provides a strong correctness check without adding a production MILP dependency.

17. Suggested implementation order

Phase 1: Model and validation

1. Fix the package import in `models.py` to use `.loss_models`.
2. Add `LossSegment`.
3. Add constant-loss normalization.
4. Add date resolution.
5. Add curve and delivered-flow validation.
6. Add dedicated unit tests for `loss_models.py`.

Phase 2: Fixed-driver piecewise curves

1. Add compiled `LossSite`.
2. Replace direct arithmetic on `LossDefinition`.
3. Evaluate curves from fixed daily measured or residual flows.
4. Preserve all existing constant-loss tests.
5. Add nonzero-intercept tests.

Phase 3: Aggregate active-segment LP

1. Add total driver and total loss expressions.
2. Add active segment equation and guard bounds.
3. Add `ActiveLossState`.
4. Add dynamic constraint-update methods.
5. Implement initial feasible baseline capture.

Phase 4: Event-driven allocations

1. Wrap `_maximize_var()`.
2. Wrap proportional-group maximization.
3. Detect and process loss-boundary events.
4. Add progress and final commitment semantics.
5. Add cycle detection.
6. Add loss-event evidence.

Phase 5: Transaction-specific loss attribution

1. Replace dynamically injected loss-route path items with dedicated loss allocation variables.

2. Add baseline-relative marginal loss equations.
3. Sum transaction losses to aggregate physical loss.
4. Add transaction-level reporting.

Phase 6: Dynamic natural-flow integration

1. Replace fixed precomputed loss-adjusted natural-flow bounds.
2. Recalculate or relinearize natural-flow availability after events.
3. Apply the same state engine in both solve phases.
4. Test diversions, imports, storage releases, and spill reallocation together.

Phase 7: Performance work

1. Measure the number of LP solves per schedule item.
2. Avoid solving again when no curve driver can move.
3. Batch simultaneous breakpoint events.
4. Consider switching from the SciPy wrapper to `highspy` for basis reuse.
5. Add diagnostics for solve count, iterations, and segment transitions.

18. Acceptance criteria

The implementation is complete when:

1. Existing constant-fraction tests produce equivalent results.
2. Every solved loss site satisfies:

$$|L_{LP} - L_{\text{curve}}(Q_{LP})| \leq \text{tolerance}$$

1. No allocation crosses a loss breakpoint while using coefficients from the previous segment.
 2. A breakpoint with an adjacent segment never becomes the final limiting reason.
 3. Senior transaction allocations remain fixed while junior transactions are solved.
 4. Proportional groups maintain the required proportions across segment changes.
 5. Time-varying loss definitions are resolved independently for each day.
 6. Losses are physically balanced on the configured loss routes.
 7. The output records every segment transition and the final real limiting constraint.
 8. Ambiguous 100%-loss and reverse-flow configurations fail with clear validation messages.
-

19. Most important architectural rule

Do not implement variable losses by merely replacing:

`loss_fraction`

with:


```
active_segment.slope
```

inside the current `rem_factor` calculation.

That would discard the segment intercept and would retroactively apply the new marginal loss rate to water allocated under previous segments.

The safe formulation is:

- An aggregate affine loss equation for physical correctness.
- Baseline-relative incremental equations for transaction continuity and attribution.
- Segment guard constraints.
- An inner event loop that continues the same priority objective after each breakpoint.